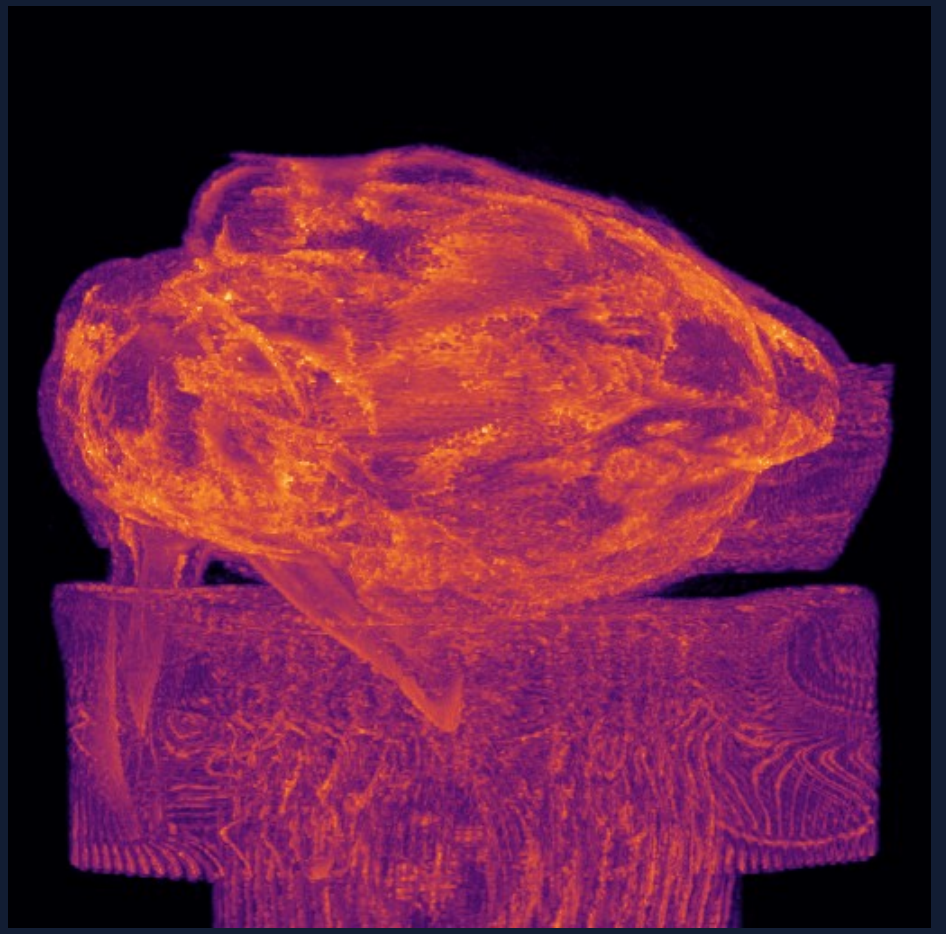
LARGE-SCALE COMPUTING · PRACTICAL ASSIGNMENT 3

BunnyMIP

Volumetric Maximum Intensity Projection with CUDA

A GPU port of a three-stage volumetric rendering pipeline — thresholding · 3×3×3 Gaussian blur · rotated MIP — validated against a CPU gold standard and benchmarked.

Group cle08 · Hugo Castro (113889) · Ângela Ribeiro (109061) · Luís Diogo (108668)



Maximum Intensity Projection of the Stanford Bunny CT volume

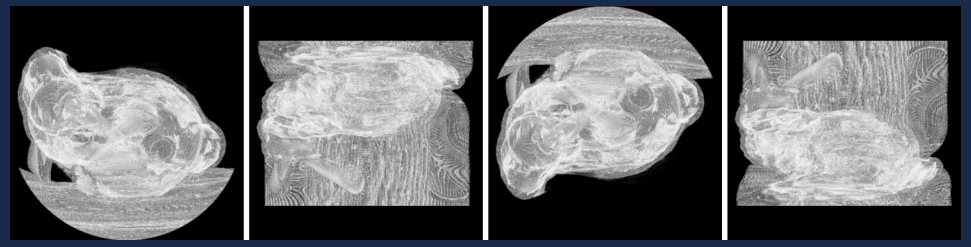
The problem: turning a CT volume into an image

- **Input:** Stanford Bunny CT scan
- **512 × 512 × 361 voxels** uint16 density field, ≈ 180 MiB
- **≈ 9.5 × 10^7 voxels** processed every render
- **Output:** a single 512 × 512 projected image

THE CHALLENGE

The sequential CPU pipeline takes **~6.8 seconds** per frame. Goal: port the three stages to the GPU with CUDA and quantify the speedup.

The bunny, projected at four rotation angles



Maximum Intensity Projection reveals internal density

A three-stage pipeline, ported stage by stage

1 THRESHOLD

Point operation

$v < T \Rightarrow 0$

remove air noise

2 GAUSSIAN BLUR

3×3×3 stencil

weighted average of
27 neighbours

3 MIP

Ray reduction

max intensity along
each rotated ray

DATA LAYOUT contiguous 1-D uint16 array · $idx = z \cdot N^2 + y \cdot N + x$ (N = 512, x is the fastest-varying axis)

Same threshold, σ and rotation matrix R feed both the CPU and GPU pipelines — which is what makes the validation meaningful.

Device memory model

GLOBAL MEMORY — three device buffers

`d_volume`
512×512×361 uint16
thresholded in place

`d_blurred`
512×512×361 uint16
blur output — out-of-place

`d_raster`
512×512 uint16
final MIP image

CONSTANT MEMORY read-only · broadcast

`c_weights` — 27 Gaussian weights

`c_R` — flattened 3×3 rotation matrix

WHY THESE CHOICES

- **Constant cache broadcasts** every thread reads the same R / weights in one transaction
- **Weights computed on the host** the costly exp / normalisation runs once, not per thread
- **Out-of-place blur** separate output buffer avoids read-after-write hazards
- **__ldg read-only cache** volume reads in the MIP go through the texture path
- **VRAM freed each run** no large allocations left resident — server etiquette

Thresholding — the point operation

POINT OP

- **Embarrassingly parallel** one thread handles many voxels
- **Grid-stride loop** decouples launch size from data size
- **Fully coalesced** consecutive threads → consecutive addresses
- **256 threads / block** over 9.5×10^7 voxels

device-bunnyMIP.cu

```
__global__ void threshold_kernel(
    uint16_t* v, int N, uint16_t T){
  for (i = tid; i < N; i += stride)
    if (v[i] < T) v[i] = 0;
}
```

In place on `d_volume`. Mirrors the host exactly — same comparison, same result.

Gaussian blur — the 3×3×3 stencil

STENCIL

- **One thread per voxel** organised as 8×8×8 blocks on a 3-D grid
- **27-tap convolution** weighted by c_weights from constant memory
- **Clamped boundaries** mirrors the host's max/min edge handling
- **Out-of-place + __restrict__** no write hazards; enables aliasing opts

device-bunnyMIP.cu

```
__global__ void blur_kernel(
  const uint16_t* __restrict__ in,
        uint16_t* __restrict__ out){
  float acc = 0;
  for (27 neighbours k)
    acc += c_weights[k]*in[clamp(idx)];
  out[i] = (uint16_t)acc;
}
```

Truncating `(uint16_t)` cast reproduces the host's arithmetic bit-for-bit.

Maximum Intensity Projection — the reduction

RAY-CAST

- **One thread per output pixel** organised as 16×16 blocks
- **Backward mapping** screen → volume via c_R, nearest-neighbour
- **ray_range = 405** half-diagonal → 810 steps span the full diagonal
- **No atomics, no sync** each ray's maximum is fully private

device-bunnyMIP.cu

```
__global__ void mip_kernel(...){
  uint16_t best = 0;
  for (t = -range; t < range; ++t){
    p = c_R * ray(x,y,t) + offset;
    if (in_bounds(p))
      best = max(best, __ldg(&vol[p]));
  }
  raster[y*N+x] = best;
}
```

Volume read through **`__ldg`** (read-only data cache) — the kernel never writes the volume.

Concurrency strategy at a glance

STAGE	MAPPING	KEY TECHNIQUE
Threshold	1 thread ↔ many voxels	**grid-stride loop**
Blur	1 thread ↔ 1 voxel	**independent reads, single input**
MIP	1 thread ↔ 1 output pixel	**private ray reduction**

BY CONSTRUCTION

No locks · no atomics · no race conditions. Read-only inputs live in constant memory; the volume is read through the read-only cache.

Validated against the CPU gold standard

0.00 %

differing pixels

across every benchmarked configuration

0

max absolute difference

16-bit intensity units

PASS

tolerance ≤ 1.0 %

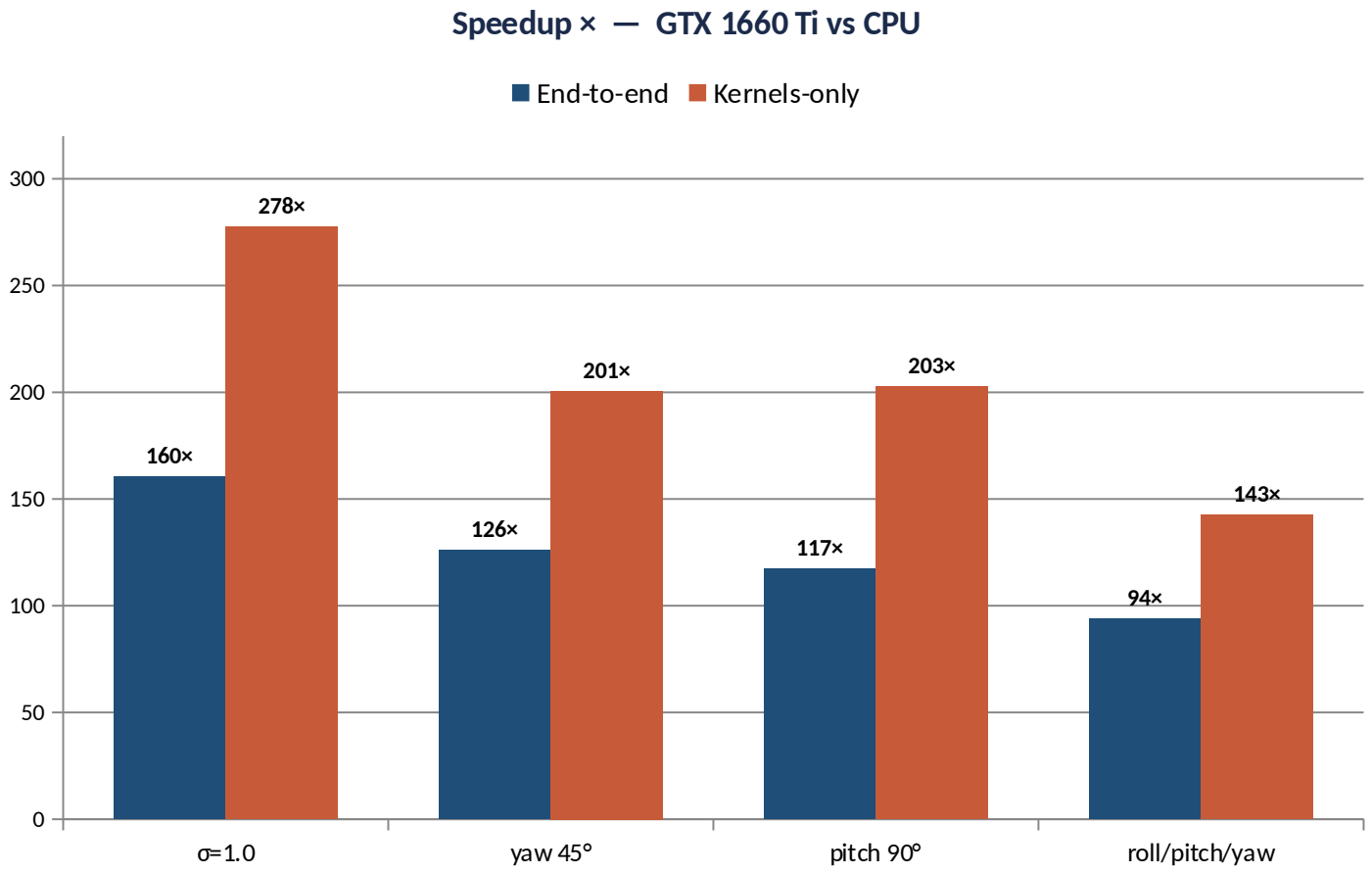
non-zero exit code on fail — scriptable

WHY THE OUTPUT IS BIT-STABLE

The kernels reproduce the host's arithmetic exactly — same weight math, same boundary clamping, the same (uint16_t) truncation, the same nearest-neighbour ray sampling and the same ray range.

Metric: percentage of pixels whose absolute difference exceeds PIXEL_TOL = 2.

Speedup over the sequential CPU



HEADLINE

93–165× end-to-end

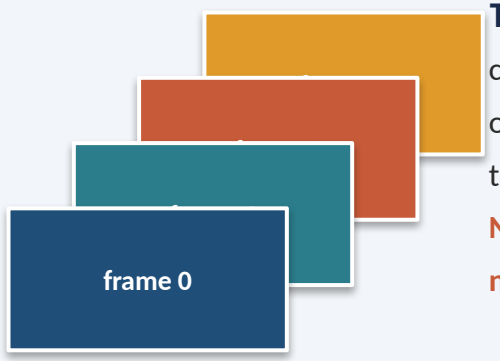
143–287× kernels

- **CPU baseline ≈ 6.8 s** GPU end-to-end ≈ 42 ms
- **Blur is σ-independent** same 27-tap work for any σ
- **~18 ms fixed overhead** 180 MiB H→D upload + alloc

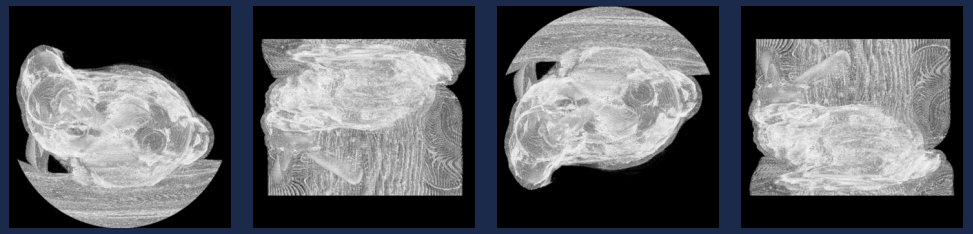
N MIP frames in a single kernel call

- **Threshold + blur run once** the blurred volume is reused for every frame
- **3-D grid (x, y, frame)** grid z-dimension — one layer per frame
- **One launch renders all N MIPs** each frame uses its own c_Rmulti[f]
- **N matrices in constant memory** MAX_FRAMES = 512 ≈ 18 KB
- **Encoded into a rotating video** FFmpeg, contrast-normalised to 8-bit

3-D GRID — frame axis as extra layers



`Thread (x, y, f)`
casts pixel (x, y)
of frame f through
the shared volume.
No atomics,
no sync.



`output/frame_000.pgm … frame_119.pgm → bunny-spin.mp4`

SUMMARY

Three kernels, one validated GPU pipeline

Faithful port

Bit-stable with the CPU gold standard — 0.00 % difference

Large speedup

93–165× end-to-end, up to 287× kernels-only

Sound CUDA design

Constant memory · grid-stride · __ldg · no atomics

Innovation

N MIP frames in a single kernel call → rotating video